

# Operating Systems Lab

## Assignment - 5

### ❖ Process Synchronization using xv6

#### Instructions for Students

- All implementations must be done inside the **xv6 source tree** (add/modify files under `kernel/`, `user/`, and update `Makefile` as needed).
- You may use xv6's existing `sleeplock.c/spinlock.c` primitives, or implement your own synchronization primitive (semaphore/mutex) as a **new system call** where a question asks you to.
- Submit:
  1. All modified/added source files (`.c`, `.h`).
  2. Updated `Makefile` (if new user programs/syscalls added).
  3. A short **README** explaining your design and how to build/run (`make qemu`).
  4. A **screenshot/terminal log** showing correct output for each question.
- Add meaningful comments in code. Marks will be deducted for uncommented or unexplained logic.
- Plagiarism between groups will result in a zero for all involved.

#### Question 1 : Mutual Exclusion using Peterson's Algorithm

Implement **Peterson's Algorithm** to achieve mutual exclusion between **two processes** sharing a critical section, using only shared memory variables (no OS-provided locks allowed for the actual mutual exclusion logic — the point is to enforce it purely through Peterson's `flag[]` and `turn` variables).

##### Requirements:

1. Since xv6 processes do not share memory by default, first set up a **shared memory region** between a parent and one child process. You may do this by:
  - Adding a simple shared-page mechanism in the kernel (e.g., a new syscall `int shm_get(void)` that returns a pointer to a page shared between a parent and its child after `fork()`), **or**
  - Any other method of your choice — clearly document and justify your approach in the README.
2. In this shared memory, declare the two Peterson's Algorithm variables:

- `int flag[2]` — intent of each process to enter the critical section.
  - `int turn` — whose turn it is to enter when both want in.
3. Write a user-level program (`peterston.c`) that:
    - Forks one child (process 0 = parent, process 1 = child).
    - Each process runs a loop (e.g., 10 iterations) where it:
      - Sets its `flag[i] = 1`, sets `turn` to the other process, then busy-waits while `flag[other] == 1 && turn == other`.
      - Enters the critical section: increments a shared counter `shared_counter` and prints "Process <id> in CS, counter = <value>".
      - Exits the critical section by setting `flag[i] = 0`.
      - Does some "remainder section" work (e.g., a short delay loop) before repeating.
  4. Verify correctness by checking:
    - `shared_counter` reaches exactly the expected final value (no lost updates).
    - No two "in CS" prints from different processes are ever interleaved without an exit in between.

**Deliverable:** Kernel/shared-memory setup + `peterston.c` + output log demonstrating correct mutual exclusion and a correct final counter value.

## Question 2 : Producer-Consumer Problem (Bounded Buffer)

Implement the classic **Producer-Consumer problem** using a fixed-size circular buffer, entirely within xv6.

### Requirements:

1. Use **two counting semaphores** (`empty` and `full`) and **one mutex/lock** to protect the shared buffer. You must implement your own lightweight semaphore for this question (e.g., via new syscalls backed by xv6's `sleep()/wakeup()`, or by using xv6's existing spinlock/sleeplock primitives directly) — clearly document your implementation in the README.
2. Implement this as a **single user-space program** (`prodcons.c`) that:
  - Uses `fork()` to create one producer process and one consumer process sharing a buffer via a memory region (you may use a shared page through xv6's memory model, or simulate shared state using pipes if true shared memory is not available — clearly state and justify your chosen approach).
  - The buffer size should be configurable (default = 5).
  - Producer produces integers 1 to 20 and inserts them into the buffer, sleeping briefly between insertions.

- Consumer removes items from the buffer and prints them, sleeping briefly between removals.
3. Demonstrate that:
- The producer blocks when the buffer is full.
  - The consumer blocks when the buffer is empty.
  - No item is lost, duplicated, or read out of order.

**Deliverable:** Source code + explanation of how shared state was achieved in xv6 + output log showing correct blocking/unblocking behavior.

## Question 3 : Readers-Writers Problem

Implement the **first Readers-Writers problem** (readers have priority, but no reader should starve writers indefinitely for full credit).

### Requirements:

1. Use synchronization primitives (xv6 spinlocks/sleeplocks, or your own semaphore implementation) to protect a shared integer variable `shared_data`.
2. Implement `readwrite.c` with:
  - A `read_count` variable tracking the number of active readers, protected by a mutex.
  - Multiple **reader processes** that read `shared_data` and print its value, allowing concurrent reads.
  - Multiple **writer processes** that increment `shared_data` and print the new value, requiring exclusive access.
3. Spawn at least **3 readers and 2 writers** using `fork()`.
4. Show through output logging that:
  - Multiple readers can access `shared_data` simultaneously.
  - A writer gets exclusive access (no reader or other writer active during a write).

**Deliverable:** Source code + logic explanation (readers-priority vs fair solution) + output log with timestamps/PIDs showing correct interleaving.

## Question 4 : Dining Philosophers Problem (Deadlock Avoidance)

Implement the **Dining Philosophers problem** with 5 philosophers and 5 forks (chopsticks) in xv6, ensuring your solution is **deadlock-free**.

### Requirements:

1. Represent each fork as a binary semaphore/lock (using xv6 spinlocks/sleeplocks or your own semaphore implementation).
2. Implement `dining.c` where each philosopher is a separate process (`fork()` × 5), cycling through: **thinking** → **hungry** → **eating** → **thinking**, indefinitely or for N cycles (configurable, default N = 5 per philosopher).
3. Apply **at least one deadlock-avoidance strategy**, such as:
  - Resource ordering (pick lower-numbered fork first), **or**
  - Allowing at most 4 philosophers to sit at the table at once, **or**
  - An asymmetric solution (odd philosophers pick left fork first, even pick right fork first).
  - Clearly state which strategy you used and why it prevents deadlock.
4. Print state transitions for each philosopher, e.g.: **Philosopher 2: THINKING -> HUNGRY -> EATING -> THINKING**
5. Run the program long enough to demonstrate no deadlock occurs (no permanent hang) and reasonable fairness (no philosopher starves for the full test duration).

**Deliverable:** Source code + a short write-up (half a page) identifying the deadlock-avoidance technique used and why it works + output log confirming all philosophers complete their cycles.

## Submission Format

- Folder name: `RollNumber_Name_Assignment4`
- Contents: `/kernel` changes (diff or full files), `/user` programs, `Makefile`, `README.md`, `output_logs/` (screenshots or `.txt` transcripts from `qemu`).
- Compress as `.zip` or `.tar.gz` and submit via the LMS before the deadline.

**Deadline: 07 September 2026**